Esame Piattaforme Cloud
Manuela Zanni matr. 0522700020

Progetto: Architettura cloud per object detection

Tecnologie: OpenStack (via DevStack), YOLOv11, Python 3.x, Streamlit.

Architettura del sistema

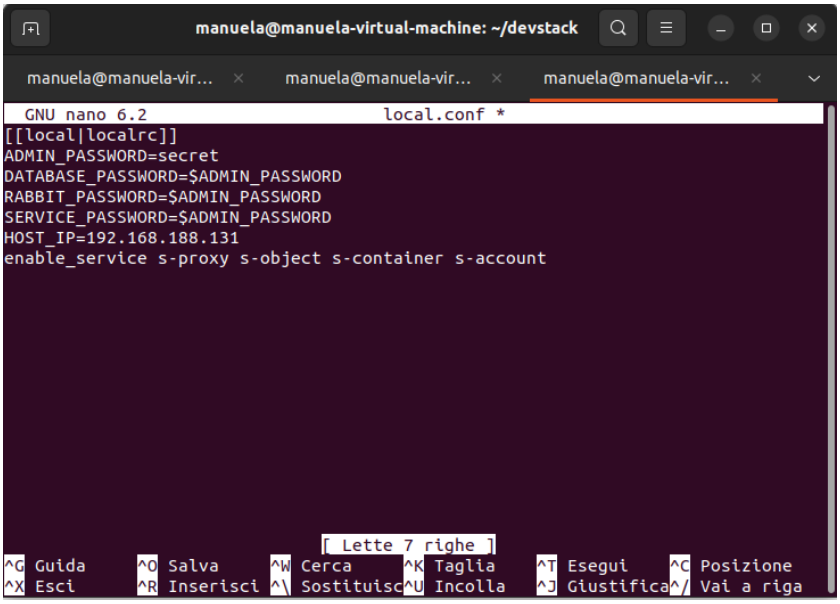
Il sistema si basa su un'infrastruttura **IaaS (Infrastructure as a Service)** gestita tramite Openstack. L'approccio utilizzato è il disaccoppiamento dei servizi:

- **Image Service (Glance):** serve da archivio centralizzato e immutabile per i modelli di machine learning.
- **Object Storage (Swift):** archiviazione persistente dei risultati, organizzata in container.
- **Identity Service (Keystone):** gestisce l'autenticazione tramite token-based access.

Fase di installazione e provisioning

Setup DevStack

L'installazione è stata eseguita su una **VM Ubuntu 22.04 LTS.** È stato effettuato il download dal repository git clone *https://opendev.org/openstack/devstack* ed è stato configurato il file **local.conf** nella directory **~/devstack$** per abilitare i servizi minimi necessari.



Infine, è stato eseguito il comando di avvio *./stack.sh* per procedere con l'installazione.

Attraverso il comando *openstack hypervisor list*, è stato verificato che il nodo fosse attivo con hypervisor **QEMU**, garantendo le capacità di virtualizzazione necessarie per l'inferenza AI.

```
manuela@manuela-virtual-machine:~/progetto_ia$ openstack hypervisor list
+--------------+---------------------+-----------------+-----------------+-------+
| ID           | Hypervisor Hostname | Hypervisor Type | Host IP         | State |
+--------------+---------------------+-----------------+-----------------+-------+
| b4db89db-cb7c- | manuela-virtual-    | QEMU            | 192.168.188.131 | up    |
| 48f5-98ac-   | machine             |                 |                 |       |
| 3832be948ab3 |                     |                 |                 |       |
+--------------+---------------------+-----------------+-----------------+-------+
```

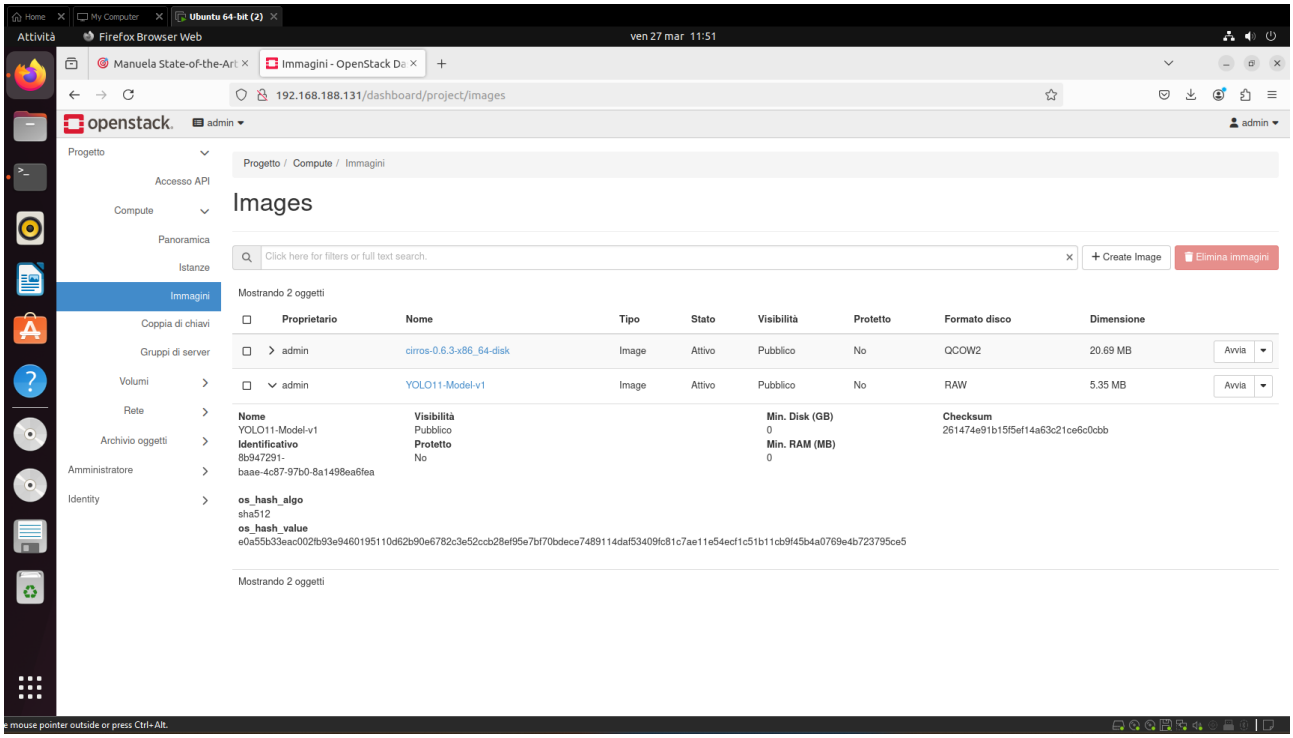
Implementazione del modello AI (Glance)

Invece di un caricamento locale statico su VM, si è scelto di utilizzare l'image service Glance di Openstack come repository del modello di AI **YOLOv11**, in modo tale che se il modello viene aggiornato, è sufficiente aggiornare l'immagine su Glance affinché tutte le istanze collegate ricevano l'aggiornamento.

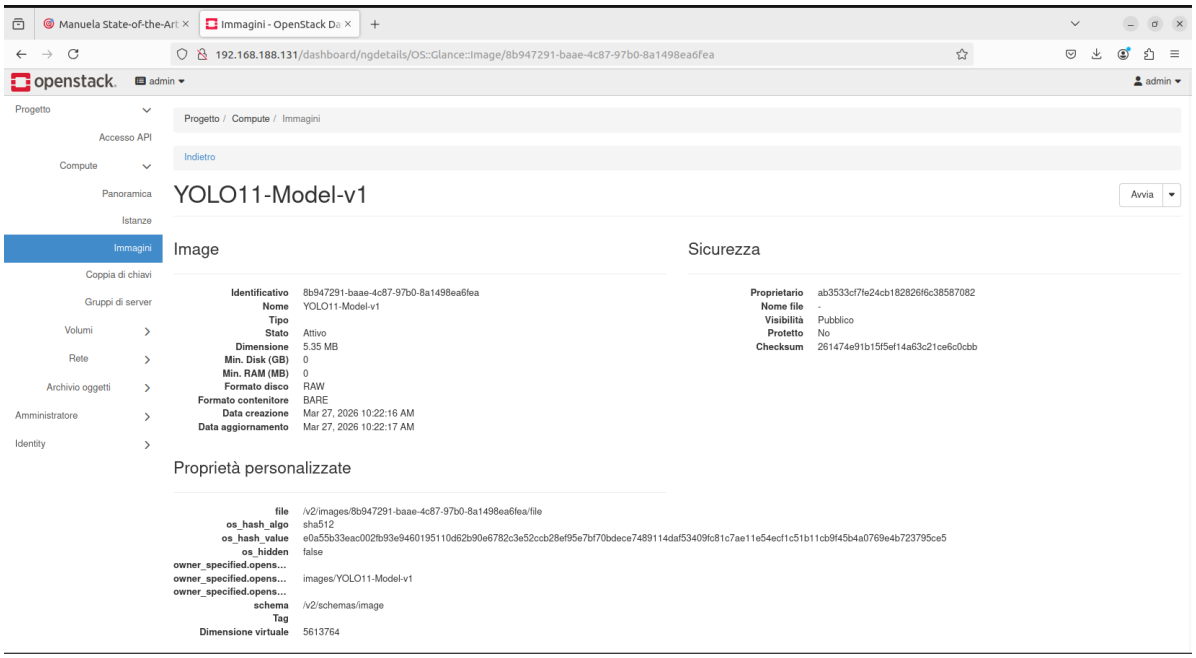
In uno scenario di produzione con centinaia di nodi di calcolo, Nova può utilizzare Glance per distribuire il modello velocemente a ogni nuova istanza creata.

Nel caso specifico di questo progetto, inizialmente era stato scelto di utilizzare il modello YOLOv3, poi sostituito con la v11.

Il file YOLO11-Model-v1 è stato caricato come immagine con formato disco raw e contenitore bare.



Esame Piattaforme Cloud
Manuela Zanni matr. 0522700020




```
manuela@manuela-virtual-machine:~$ openstack image list
+--------------------------------------+--------------------------+--------+
| ID                                   | Name                     | Status |
+--------------------------------------+--------------------------+--------+
| 8b947291-baae-4c87-97b0-8a1498ea6fea | YOLO11-Model-v1          | active |
| 6f44f8a1-7c8e-4417-a794-fcfce377d8a7 | cirros-0.6.3-x86_64-disk | active |
+--------------------------------------+--------------------------+--------+
```

L'identificativo univoco 8b947291-baae-4c87-97b0-8a1498ea6fea permette agli script di richiamare il modello in modo univoco all'interno dell'intera infrastruttura, indipendentemente dal nome assegnato. Per assicurare l'integrità dei dati, il sistema ha generato un checksum MD5 (261474e91b15f5ef14a63c21ce6c0cbb) e un hash SHA-512. L'uso degli algoritmi di hashing garantisce che il modello AI utilizzato per l'inferenza non sia stato corrotto durante il caricamento o manomesso. Infine, i parametri Min. Disk: 0 e Min. RAM: 0 dimostrano che Glance tratta il file come un Blob non come un'istanza avviabile. Questo ottimizza lo storage, occupando solo lo spazio effettivo del file .pt senza riservare risorse di virtualizzazione inutilizzate.

La logica di gestione del modello AI implementata negli script app_ia.py e cli_ia.py segue un paradigma di allineamento costante tra il repository centrale cloud e il nodo computazionale locale.

Per evitare download ridondanti e garantire che il file locale sia integro, lo script, tramite le API di Glance, interroga il servizio per ottenere il checksum del file memorizzato nel cloud. Lo script calcola l'MD5 del file presente sul disco locale (YOLO11-Model-v1.pt) e se gli hash corrispondono, il file locale è considerato valido e aggiornato e il processo di analisi delle immagini parte istantaneamente. Se gli hash differiscono o il file manca, il sistema rileva un disallineamento. In questo caso, viene avviata una procedura di sincronizzazione. Viene aperta una connessione stream verso Glance, il modello viene scaricato, sovrascrive la versione locale e viene aggiunta l'estensione .pt al nome del file locale per garantire la piena

compatibilità con il framework Ultralytics YOLO, che richiede esplicitamente l'estensione per il riconoscimento del formato PyTorch.

```python
# --- CONFIGURAZIONE MODELLO ---
MODEL_NAME_ON_GLANCE = "YOLO11-Model-v1"
LOCAL_FILE_PATH = "YOLO11-Model-v1.pt"

def get_local_md5(filename):
    if not os.path.exists(filename):
        return None
    hash_md5 = hashlib.md5()
    with open(filename, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b""):
            hash_md5.update(chunk)
    return hash_md5.hexdigest()

def main():
    if len(sys.argv) < 2:
        print("Uso: python3 cli_ia.py /percorso/immagine.jpg")
        sys.exit(1)

    image_path = sys.argv[1]

    try:
        # Autenticazione
        auth = v3.Password(auth_url=AUTH_URL, username=USERNAME, password=PASSWORD,
                           project_name=PROJECT_NAME, user_domain_name=USER_DOMAIN_NAME,
                           project_domain_name=PROJECT_DOMAIN_NAME)
        sess = session.Session(auth=auth)
        glance = GlanceClient('2', session=sess)
        swift = swift_client.Connection(session=sess)

        # Sincronizzazione modello (Glance)
        remote_image = next(glance.images.list(filters={'name': MODEL_NAME_ON_GLANCE}))
        if get_local_md5(LOCAL_FILE_PATH) != remote_image.checksum:
            print(f"Aggiornamento modello da Glance...")
            with open(LOCAL_FILE_PATH, 'wb') as f:
                for chunk in glance.images.data(remote_image.id):
                    f.write(chunk)
```

Sviluppo del software di analisi

Il software è diviso in due moduli che condividono la logica di connessione verso Openstack.

```
manuela@manuela-virtual-machine:~/progetto_ia$ tree ~/progetto_ia
/home/manuela/progetto_ia
├── app_ia.py
├── cli_ia.py
└── YOLO11-Model-v1.pt

0 directories, 3 files
```

La struttura della cartella **~/progetto_ia** è organizzata secondo i tre componenti fondamentali del sistema:

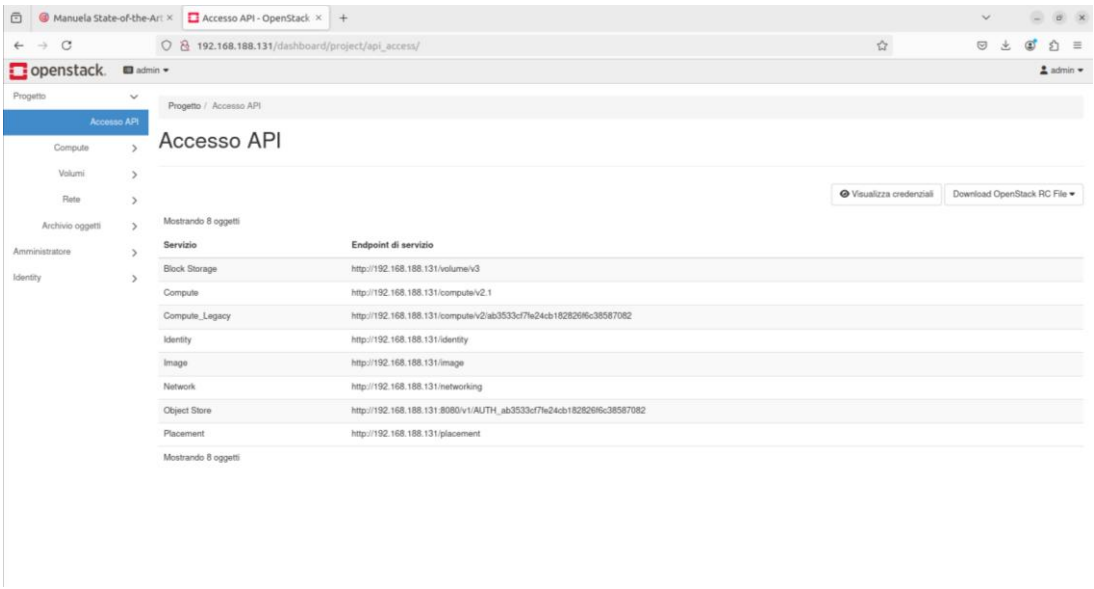
- **app_ia.py:** è il modulo dedicato all'utente finale e utilizza il framework Streamlit per creare una dashboard interattiva. Serv per gestire l'upload delle immagini, l'autenticazione verso Swift e la visualizzazione grafica dei bounding boxes.
- **cli_ia.py:** è il modulo dedicato all'analisi AI in pipeline di sistema, garantendo che ogni file processato venga archiviato su Swift con il prefisso terminale_[nome_file].jpg.
- **YOLO11-Model-v1.pt:** è il file dei pesi del modello YOLOv11 (versione Nano).

Esame Piattaforme Cloud
Manuela Zanni matr. 0522700020

Modulo di connessione (Keystone e Swift)

Per permettere allo script di analisi di interagire con i servizi Openstack, è stata implementata una logica di connessione basata sulla libreria ufficiale python-swiftclient. Il dizionario di configurazione *config* mappa i requisiti del servizio di identità Keystone.

```
# --- CONFIGURAZIONE OPENSTACK ---
AUTH_URL = "http://192.168.188.131/identity/v3"
USERNAME = "admin"
PASSWORD = "secret"
PROJECT_NAME = "admin"
USER_DOMAIN_NAME = "Default"
PROJECT_DOMAIN_NAME = "Default"
```



Il frammento di codice definisce l'identità del client all'interno del sistema distribuito:

- **authurl**: rappresenta l'endpoint dell'API di Keystone (/identity/v3). È l'indirizzo a cui lo script invia le credenziali per ricevere in cambio un token di accesso.
- **user e key**: rappresentano le credenziali di sicurezza (username e password). Nel progetto è stato utilizzato l'utente admin per garantire i privilegi di scrittura sui container Swift.
- **auth_version**: specifica esplicitamente l'utilizzo della versione 3 delle API, che introduce una gestione avanzata dei domini e dei progetti rispetto alle versioni precedenti.

Interfaccia web (Streamlit)

L'applicazione **app_ia.py** espone una GUI sulla porta 8501.

Il flusso prevede:

1. Upload immagine tramite widget st.file_uploader.

2. Inferenza locale tramite la classe YOLO di Ultralytics.
3. Caricamento dell'immagine elaborata su Swift.

Interfaccia CLI (Terminale)

Lo script **cli_ia.py** permette l'integrazione con pipeline automatizzate. Accetta il percorso dell'immagine come argomento da riga di comando e interagisce con Swift in modalità sincrona.

Storage e gestione dati (Swift)

I dati vengono organizzati nel container **immagini_ia.**

```
manuela@manuela-virtual-machine:~/devstack$ openstack container list
+-------------+
| Name        |
+-------------+
| immagini_ia |
+-------------+
```

```
manuela@manuela-virtual-machine:~/progetto_ia$ openstack object list immagini_ia
+-----------------------------------+
| Name                              |
+-----------------------------------+
| /home/manuela/Scrivania/cane.jpg  |
| UI_foto_1.jpg                     |
| UI_foto_10.jpg                    |
| UI_foto_11.jpg                    |
| UI_foto_12.jpg                    |
| UI_foto_4.jpg                     |
| UI_foto_5.jpg                     |
| UI_foto_6.jpg                     |
| UI_foto_7.jpg                     |
| UI_foto_8.jpg                     |
| UI_foto_9.jpg                     |
| terminale_foto_10.jpg             |
| terminale_foto_11.jpg             |
| terminale_foto_8.jpg              |
+-----------------------------------+
```

Ogni oggetto salvato segue una naming convention specifica per distinguere la sorgente:

- UI_[nome_file].jpg → Immagini processate in modalità UI
- terminale_[nome_file].jpg → Immagine caricata tramite script CLI

Questa struttura permette di analizzare i log di accesso e verificare il carico di lavoro distribuito tra le diverse interfacce di input.

Modalità di esecuzione

Il sistema è stato progettato per offrire flessibilità d'uso attraverso due interfacce distinte, che condividono la medesima logica di backend basata su Openstack e il modello YOLOv11.

Esecuzione tramite interfaccia CLI

L'interfaccia a riga di comando è dedicata a operazioni rapide.

Viene invocato lo script passando come argomento il percorso locale dell'immagine (es. *python3 cli_ia.py ~/Scrivania/foto_1.jpg*). Lo script estrae il nome del file tramite *os.path.basename*, si autentica su Keystone e carica l'immagine originale su Swift con il prefisso *terminale_[nome_file].jpg*. Successivamente, avvia l'inferenza locale.

```
manuela@manuela-virtual-machine:~/progetto_ia$ python3 ~/progetto_ia/cli_ia.py ~
/Scrivania/foto_1.jpg

Analisi AI in corso su: foto_1.jpg
------------------------------
OGGETTO          | CONFIDENZA
------------------------------
dog              | 94.50%
sports ball      | 49.10%
------------------------------
Risultato caricato su Swift nel container 'immagini_ia'
```

I risultati (classe dell'oggetto e confidenza) vengono stampati direttamente nel terminale in formato testuale.

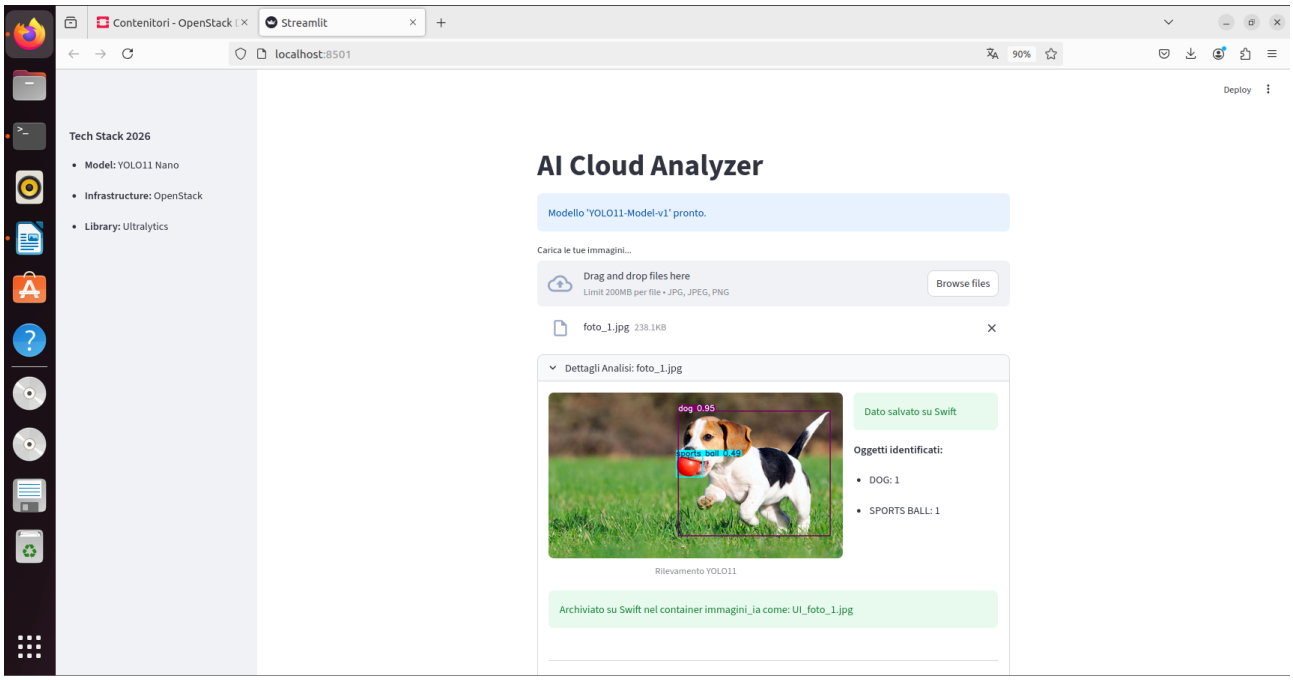
Esecuzione tramite Web UI

L'interfaccia grafica, sviluppata con il framework Streamlit, è ottimizzata per l'utente finale e l'interazione in tempo reale.

L'applicazione viene avviata sul server con il comando *streamlit run app_ia.py*, rendendosi accessibile via browser sulla porta 8501 (*http://localhost:8501*).

```
manuela@manuela-virtual-machine:~/progetto_ia$ python3 -m streamlit run app_ia.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.188.131:8501


0: 384x640 1 dog, 1 sports ball, 64.4ms
Speed: 2.8ms preprocess, 64.4ms inference, 1.1ms postprocess per image at shape (1, 3, 384, 640)
2026-03-27 11:08:14.655 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.

0: 384x640 1 dog, 1 sports ball, 51.4ms
Speed: 1.7ms preprocess, 51.4ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)
2026-03-27 11:08:32.183 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
```

L'utente carica un'immagine tramite drag-and-drop. Lo script gestisce il caricamento del file in memoria, effettua l'autenticazione verso i servizi Cloud e salva il file nel container Swift con prefissi identificativi (es. UI_[nome_file].jpg).



A differenza della CLI, la Web UI mostra visivamente l'immagine con i bounding boxes sovrapposti agli oggetti identificati, fornendo un feedback immediato e intuitivo.

Indipendentemente dall'interfaccia utilizzata, l'esecuzione converge sempre verso lo storage centralizzato su Swift. Come dimostrato dall'output del comando *openstack object list immagini_ia*, il container funge da archivio storico unificato, dove la coesistenza di file con prefissi diversi testimonia l'interoperabilità tra le due modalità di esecuzione. Questa architettura garantisce che un'immagine caricata via CLI sia immediatamente disponibile per il controllo da parte di un amministratore tramite la dashboard di Openstack, assicurando la persistenza del dato oltre la durata della sessione di esecuzione.